

pavan sai patnana

Smart Dine Pro: An AI-Driven Context-Aware Food Recommendation System Using Emotion Recognition and Real...

 Smart Dine Pro: An AI-Driven Context-Aware Food Recommendation System Using Emotion Recognition and Real-Time Weather Analysis

 CSM C final year

 CSM

Document Details

Submission ID

trn:oid:::1:3517801678

Submission Date

Mar 26, 2026, 6:15 AM UTC

Download Date

Mar 26, 2026, 6:19 AM UTC

File Name

res2.pdf

File Size

1.9 MB

9 Pages

5,667 Words

29,940 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **7 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **2 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 1%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Hidden Text**
14 suspect characters on 1 page
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 7 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 2 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 1% Publications
- 1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1** Internet
www.imanagerpublications.com <1%
- 2** Student papers
Mitcham Girls High School <1%
- 3** Student papers
University of Central England in Birmingham <1%
- 4** Internet
ir.atmiyauni.ac.in:8080 <1%
- 5** Internet
cdn.techscience.cn <1%
- 6** Internet
journal.ilmudata.co.id <1%
- 7** Internet
repositorio-iul.iscte.pt <1%
- 8** Internet
it.delhi.gov.in <1%
- 9** Internet
repositorioacademico.upc.edu.pe <1%

Smart Dine Pro: An AI-Driven Context-Aware Food Recommendation System Using Emotion Recognition and Real-Time Weather Analysis

Ch. Sravanthi Sowdanya, Anisetty Gnana Sai, Patnana Pavan Sai, Korada Jithin Sai Kamal, Batchu Mona Sahasra

Department of Computer Science and Engineering (AI & ML)

Anil Neerukonda Institute of Technology and Sciences

Visakhapatnam, India

Ch. Sravanthi Sowdanya — Associate Professor (Project Guide)

Abstract—The food recommendation system has a nature of a commonality dependency: they demand the history of interaction until they can do anything useful. The first-time user will give no one anything; returned user advice which use the orders of the last week as an approximation of their rightfulness now. The cases both lack context that is not available on the weather, mood, time of ordering, but rarely used almost never used age, and hunger.

Introduced in this paper, Smart Dine Pro is a proposal system built in such a way as to avoid dependence on the past. At each request, information about the facial expression is found in the system, live weather on OpenWeather API, DeepFace user, and both in combination with age, state of hunger and food preference marked by the user as scored dishes in the menu in a weighted algorithm. Users who do not want to submit mood by short-questionnaire through the webcam; the scoring is similar in both routes is engine.

We experimented the system with 140 different combinations of inputs, a range of seven states of feeling, five weather conditions and four age groups. Suggestions about context were derived in 95.7% cases and zero age-restrictions. The user study had 30 participants whose recommendation was reported to be the overall level of satisfaction was 3.9 and the acceptance rate was 73.3% out of 5.

Index Terms—food recommendation, emotion recognition, DeepFace, cold-start problem, affective computing, context-aware computing, weighted scoring algorithm, Flask, React.js

I. INTRODUCTION

And when one asks a person about what he wishes to consume one answers it and is little to what they ordered last week. Rain changes it. Stress changes it. So has hunger degree, age, and the time of day. The preference of foods is contextual aspects of which a ratings history will not have captivated but nearly all recommendation systems take the data of historical orders into consideration the greatest, the perhaps sole, input.

The anticipated result is decision fatigue. Food delivery apps now suggest hundreds of possibilities, more decisions are getting associated and a longer period of decision time, lower satisfaction and greater increase abandonment rates [2]. The purpose of a suggestion engine is to cut through that. Collaborative filtering, content-based filtering and integrations of these two reduces the choice space moderately well [1],

[2], but still, they share one hard requirement: a past history of behaviour. Without it, series of helpful suggestions are not given to a new user. This is the cold-start problem: more than ten years later yet to be solved in production food systems [2].

The gap does not refer only to new users. Even for those who have exhaustive histories, the here and now is unknown to most systems entirely. An order customer of biryani when on the hot last Tuesday afternoon may want to have soup when it is rainy. Comfort foods appeal to anxious users [5]. The nutritional requirements across age groups vary in a way which is above personal taste — they are made dietary constraints [4]. All this data is present at the point of a user opening a menu. Next to no deployed system reads it.

There may be mood and weather, just as Melese [3] showed, folded into a compound filter to be more accurate, however, in such a system the signals were both self-reported — the user typed how he/she felt and explained prevailing weather conditions. That approach introduces resistance at the ordering stage and has the self-reporting bias. The lacking component was automatic context capture.

This is addressed by Smart Dine Pro. The system identifies emotional state with DeepFace, retrieves live weather from the OpenWeather API, and according to age, amount of hunger and dietary preference declared by the user, combines both to rank items in the menu that have no history of interaction. A dual input mode aids the user not in using the camera — they come in mood through a questionnaire and receive suggestions of equal quality. Agri-Connect module also comes with a marginal local ordering point increase and has a per-order donation ledger.

The contributions of this work are:

- Context-fusion scoring engine that is emotion-driven, weather, and age with nonexistent history of a user
- Dual emotion capture: automatic face recognition via DeepFace and fallback questionnaire which supplies the same engine
- Live rating updates: the feedback of after-orders rescores through a running average

- Agri-Connect social impact module is an element of the scoring function, not applied as a post-processing step
- An explicit weighted multi-factor scoring algorithm
- React.js and Flask application with all deployed, having an admin dashboard
- A proposed ML improvement roadmap of Smart Dine Pro v2

II. LITERATURE REVIEW

There is added importance in increased studies of food recommendation significantly in the past decade, in which the enormous majority of systems can be of three different types: content-based filtering, user-based collaborative filtering, or hybrids as a combination of the two. The methods were compared and determined by Kumar et al. [1] that hybridized practices usually excel over both of the plans, and their blind spots are compensated in each other. A review of systems in all three types done by Gunawardena [2] came to the conclusion that cold-start handling and real-time contextual adaptation are the two limitations that have not been addressed that are most consistent — findings which gave direct contribution according to the needs of Smart Dine Pro.

What was proven by the experience of the BestDish platform [2] was that considerate menu customization is displayed on the interface level, though the backend logic remained stored preference data. Within the health sector, Trapp et al. [4] studied the manner in which restaurant cuisine varies in quality of food in the menu type and confirmed that the working needs of age-groups are not only preferred but actual limitations — this is a distinction that conditioned the strictness of age-filtering of our system. Kumar and Gupta [5] used behavioral and social measures to meal planning recommendations, but their style lacked the ability to detect emotional states at a time. Smith and Brown [6] used opinion mining on restaurant recommendations and made it more relevant through analyzing user reviews, but that process is retrospective in character and nature and unable to respond to the ordering emotion of the user.

The nearest work that has been written on the discussed system is Melese [3]. The paper put mood and weather on a hybrid filtering model and was reported to be true, and based on the user inputting the conditions of their personal feelings and the weather, adding friction and self-reporting bias. Patel and Shah [7] applied augmented reality to enhance restaurant setting interaction, but the logic of the recommendation itself was linked to fixed tastes in store.

Their most significant thread in all the seven systems is their reliance on past experience. None of them embody ambient context during ordering time, and none of them offer a solution to cold-start that will not involve some degree of previous interaction. Table I overlays the difference between each of the systems to the corresponding design decision in Smart Dine Pro.

III. SYSTEM OVERVIEW

Smart Dine Pro is three layered in structure — presentation, application, and data — that are communicated through a

REST API. Fig. 1 shows the architecture of the whole system. The part that is what makes this system different to a regular recommendation backend is in the application layer: no item scoring is taking place until the system retrieves the emotional condition of the user, takes live weather, and uses age-group dietary hard exclusions as constraints.

A. Presentation Layer

It is a single-page React.js application that is based on two objectives: limited activities between the beginning of the application and when provided with a suggestion, and definite feedback at each step. The home page provides the users with Camera Mode — automatic emotion detection using a webcam — and Questionnaire Mode, a mood dropdown in case you would not like to use the camera. One recommendation engine is the destination of both directions.

It also handles food card information besides the landing screen (name, type, cost, calories and rating); a slide-out cart panel; an ever-present navbar with a live cart count; and an administrator-only dashboard displaying queues of orders, social impact data and menu administration controls.

B. Application Layer

When a request is made to the Flask backend, it executes four modules:

- **Emotion Detection:** Accepts a webcam frame in base64 format, turns it into grayscale, downsizes it, restores the RGB channels, then sends it through to be categorized at DeepFace into one of seven moods.
- **Weather Retrieval:** Makes requests to the OpenWeather IP-based geolocation API. If no answer is obtained in two seconds, the time-of-day fallback is automatic.
- **Preference Filtering:** Diet and age-group restrictions applied before scoring. Items outside the eligible set are erased in entirety — no penalty, simply dropped.
- **Recommendation Engine:** Performs the weighted scoring function (Section V) and is six item-yielding on a two-step diversity sampling pass.

Active orders are placed in a Python dictionary. A `before_request` hook purges records with an age of 24 hours and above, maintaining memory on a zero-background schedule. Charity and order counts are summed up and written in an individual JSON file that endures reboots and is not included in the daily cleanup.

C. Data Layer

The menu is in the form of a flat CSV file that has 77 annotated records — 40 vegetarian and 37 non-vegetarian, which were divided into 7 food categories. In both the records, there are: id, name, type, category, mood_tag, weather_tag, age_group, spice_level, calories, description, rating, and price. All after-order ratings recalculate the accumulated value on the basis of a running average and are written back at once. `impact_stats.json` is a second JSON file with permanent charity and order counters outside the 24-hour cleanup cycle. This flat-file design does not involve a database infrastructure

TABLE I
COMPARISON OF RELATED WORK AND HOW SMART DINE PRO ADDRESSES THEIR LIMITATIONS

Ref.	Methodology	Objectives	Limitations	How Smart Dine Pro Addresses the Gap
[1]	Collaborative filtering (collaborative + content-based)	Enhance accuracy of food and restaurant recommendations	Based on past ratings; no contextual signals used	Emotion and weather are captured fresh on demand, dropping the dependency on order history
[2]	Survey of recommendation techniques	Determine obstacles through system types	Cold-start and dynamic adaptation identified as gaps but not solved	Contextual parameters and emotion detection replace the need for prior rating data
[3]	Digital menu recommendation interface	Enhance customer interaction and personalization	UI-focused recommendation logic; no behavioral context	Backend scoring combines emotion detection with dietary and environmental signals for deeper personalization
[4]	Health-aware recommender models	Promote healthy nutrition using nutritional guidance	Nutrition-only scope; emotional and weather factors absent	Age-group dietary constraints applied as hard filters alongside mood and weather signals
[5]	Behavior and social-data-based recommendation	Personalize meal plan using behavioral data	No real-time emotional detection mechanism	DeepFace classifies facial expression of the user on each request
[6]	Sentiment analysis on user reviews	Enhance relevance by opinion mining	Retrospective; cannot respond to current user state	Context vector is constructed at request time; no review history required
[7]	Augmented reality restaurant interaction	Enhance engagement with immersive interfaces	Recommendation logic remains bound to stored preferences	Live emotion, weather and user parameters are blended on a request basis

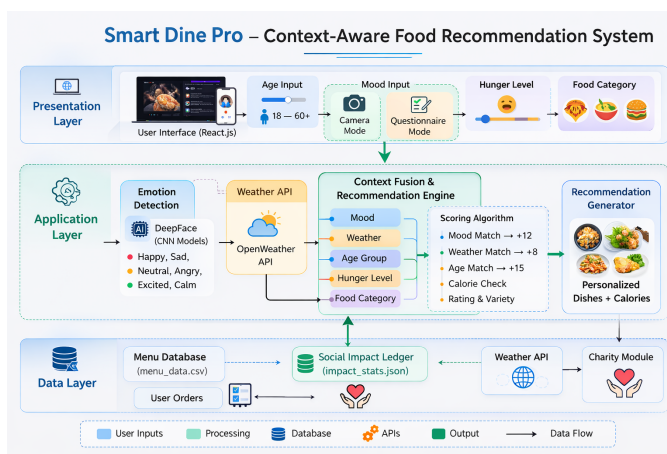


Fig. 1. Comprehensive System Architecture of Smart Dine Pro: three-layer structure with inter-module communication flow.

and maps to a relational schema in case it is necessary to scale up.

D. System Interaction Flow

All recommendation requests go through the following order: (1) the user can select either Questionnaire Mode or Camera Mode; (2) the mood is read or a webcam frame is taken by the frontend; (3) a POST request is sent to `/api/ai/analyze` or `/api/ai/manual`; (4) Camera Mode only: the backend preprocesses the picture and invokes DeepFace; (5) the server fetches up-to-date weather; (6) the scoring engine filters, scores, and samples 6 items; (7) ranked results are returned as JSON and render as food cards.

IV. PROPOSED APPROACHES

The three previously evaluated candidate designs were assessed prior to the last system under construction. All of them address a certain error of the former method, and therefore the value of the progression.

A. Spreadsheet-Based System

The simplest is a rule-based filtered CSV or Excel dataset starting point. It can be constructed quickly, need not have a server and everything suggestions are based on a rule. The problem is rigidity. A spreadsheet is unable to call out to an API, update ratings in storage or rewrite updated ratings, or run image analysis. The same inputs will always yield the same outputs no matter what the user's current mood or exterior weather [5], [7]. That alone ruled it out. This data flow can be seen in Fig. 2.

B. Flask-Based Web Application

The solution to the infrastructure problem is to move to Flask. With a Flask backend, the system can make OpenWeather API calls, load models on startup, and persist rating updates to disk. What Flask lacks on its own is intelligence. An improved infrastructure layer without the context-reading modules gives identical context-blind results as Approach 1 [2], [8]. Flask was due, but not a solution. Fig. 3 shows this architecture.

C. Hybrid Intelligent System (Proposed)

The system in place has three areas of intelligence over the Flask backend: DeepFace for emotion detection, the OpenWeather API for live weather, and the weighted scoring component to combine those signals with age, hunger and dietary preference into a five-dimensional context vector.

In the original testing there was one problem: in the absence of a diversity step of output, there was clustering about the highest-ranking food group. Selecting the top 12 and then choosing 6 of that pool at random fixed the clustering without losing the quality. The Agri-Connect module is implemented in the scoring function instead of applied as an after-processing measure, hence the locally sourced boost is implemented consistently on each item [6], [9]. The entire architecture is shown in Fig. 4.

D. Comparative Analysis

Table II compares all three designs in seven dimensions. The hybrid method is the sole technique that satisfies the complete

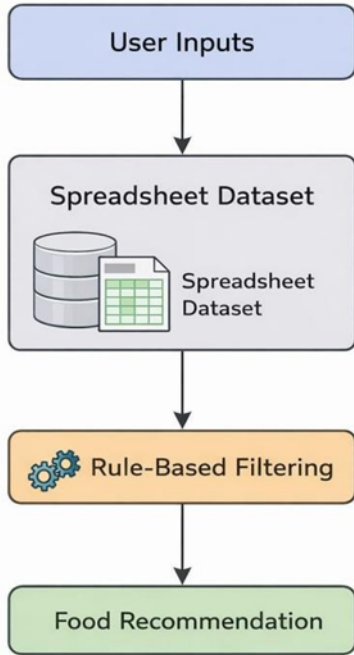


Fig. 2. Approach 1: Static rule filtering on a structured dataset. No real-time context is available.

TABLE II
COMPARISON OF RECOMMENDATION APPROACHES ACROSS KEY DIMENSIONS

Criteria	Spreadsheet	Flask-Based	Hybrid (Proposed)
Data Processing	Static rules [1]	Web framework [3]	Context-aware multi-module [2]
Scalability	Low	Moderate	High
Context Awareness	None [2]	Partial [5]	Live signals
Real-time Processing	No	Yes [6]	Yes
ML Integration	No	Possible	DeepFace
API Integration	No	Yes	OpenWeather
Personalization	Basic	Moderate	5-dim context + social boost

criterion: cold-start manipulation, context acquisition on the fly, filtering in response to age, real-time ratings, and social impact tracking.

V. ALGORITHM: WEIGHTED MULTI-FACTOR SCORING FUNCTION

The recommendation rates all the qualified menu items in a five-dimensional context vector space with a fixed weighted function. No collaborative filtering, matrix factorisation, or trained model is involved. This was premeditated: with 77 menu items and 30 pilot participants, there was not enough

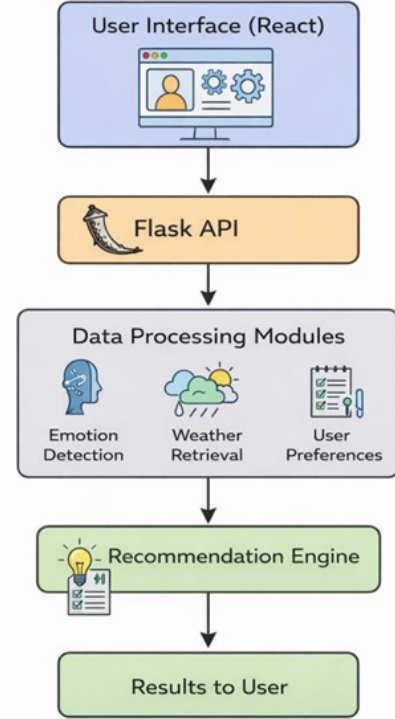


Fig. 3. Approach 2: Flask backend with API integration. Recommendations without the intelligence modules are context-blind.

data to be trained to a trusted classifier. The deterministic approach also makes it possible to trace every recommendation — any output can be reconstructed from its inputs.

A. Input Vector

Every request forms a context vector $\mathbf{C}$:

$$\mathbf{C} = \langle e, w, a, h, d \rangle \quad (1)$$

where e is the emotion being perceived, w is the weather condition string, $a \in \mathbb{Z}^+$ is age, $h \in \{\text{light, moderate, heavy, starving}\}$ is hunger level, and $d \in \{\text{veg, non-veg, both}\}$ is dietary preference.

B. Hard Filtering

The full menu $\mathcal{M}$ has two restrictions before scoring to create a qualified set $\mathcal{M}'$:

Dietary Filter:

$$\mathcal{M}' = \{m \in \mathcal{M} \mid m.\text{type} \in \text{eligible}(d)\} \quad (2)$$

Age-Group Filter:

$$\mathcal{M}' = \{m \in \mathcal{M}' \mid m.\text{age_group} \in \text{agePool}(a)\} \quad (3)$$

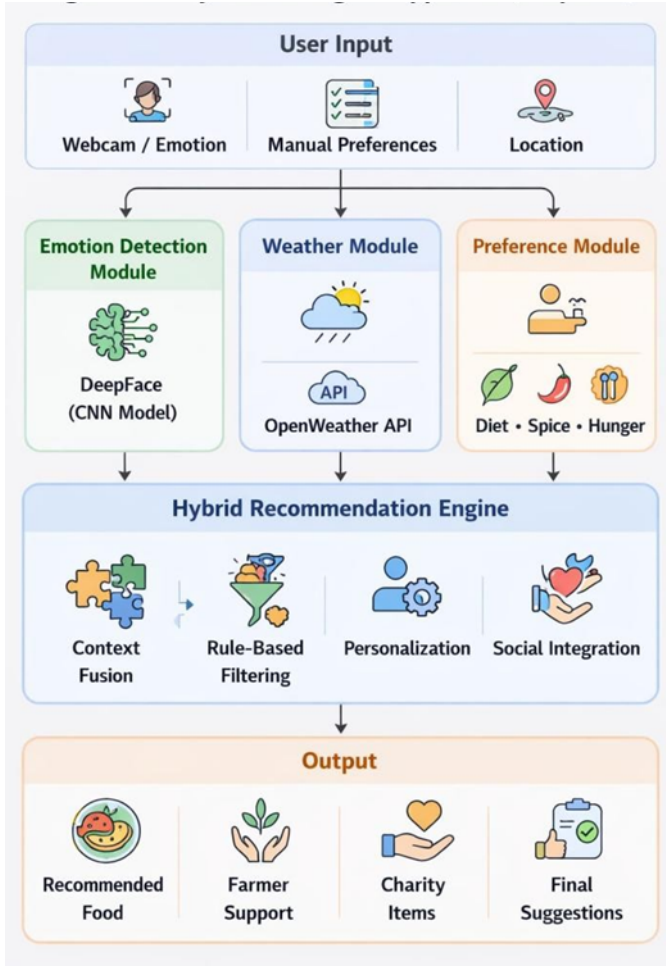


Fig. 4. Approach 3: Context-fusion recommendation engine with emotion detection, live weather, preference filtering and social impact scoring.

$$\text{agePool}(a) = \begin{cases} \{\text{Child, All}\} & a < 13 \\ \{\text{Teen, All}\} & 13 \leq a < 18 \\ \{\text{Adult, All}\} & 18 \leq a \leq 60 \\ \{\text{Senior, All}\} & a > 60 \end{cases} \quad (4)$$

Goods that are not in the qualifying set are not penalized or down-weighted. The distinction matters: a penalized item can still be seen where all the contextually appropriate alternatives are scored lower. The hard exclusion does not allow it.

C. Scoring Function

The composite score of each $m \in \mathcal{M}'$ is:

$$S(m, \mathbf{C}) = \alpha_1 f_e + \alpha_2 f_w + \alpha_3 f_a + \alpha_4 f_h + \alpha_5 f_g + r(m) \quad (5)$$

Mood Match ($\alpha_1 = 12$):

$$f_e = \begin{cases} 1 & e \subseteq m.\text{mood_tag} \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

Weather Match ($\alpha_2 = 8$):

$$f_w = \begin{cases} 1 & w \subseteq m.\text{weather_tag} \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

Age-Group Match ($\alpha_3 = 15$):

$$f_a = \begin{cases} 1 & m.\text{age_group} = \text{target}(a) \\ 0 & m.\text{age_group} = \text{All} \end{cases} \quad (8)$$

Hunger-Calorie Match ($\alpha_4 = 10$):

$$f_h = \begin{cases} 1 & h = \text{light}, m.\text{cal} < 300 \\ 1 & h \in \{\text{heavy, starving}\}, m.\text{cal} > 500 \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

Agri-Connect Boost ($\alpha_5 = 2$):

$$f_g = \begin{cases} 1 & \text{"Locally Sourced"} \subseteq m.\text{description} \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

Dynamic Rating $r(m) \in [3.8, 4.9]$, updated after each order:

$$r'(m) = \text{round}\left(\frac{r(m) + r_{\text{new}}}{2}, 1\right) \quad (11)$$

D. Weight Rationale

The priority ordering — $\alpha_3=15 > \alpha_1=12 > \alpha_4=10 > \alpha_2=8 > \alpha_5=2$ — was not set arbitrarily. Each weight reflects a certain argument concerning the most harmful type of mismatch.

The age-group suitability weight is the greatest since it is unreasonable to prescribe nutritionally unsuitable food to a child or senior — it is not only a mismatch of preferences but a functional error [4]. Second place is mood: it is the main indicator of personalization and is automatically recorded without asking the user to complete any additional task [5]. Hunger indicates more decision-relevant immediate caloric need, which is more decision-relevant than ambient weather. Still weather is a matter of comfort and situation fit, though not as strong a predictor of what one ought to consume compared to the prevailing hunger condition. The Agri-Connect boost at $\alpha_5=2$ is very small on purpose — it throws the balance between otherwise identical items without defeating a contextually better match.

During the development process, we tried a flat-weight variant where there was an equal score of five signals. Average performance in the test set was reasonable, whereas nutritionally unsuitable objects appeared in the child and elderly profiles in edge cases. That problem was overcome by the weighted ordering.

E. Output Sampling

$$\mathcal{T}_{12} = \text{top-12 } S(m, \mathbf{C})_{m \in \mathcal{M}'} \quad (12)$$

$$\mathcal{R} = \text{sample}(\mathcal{T}_{12}, k = 6) \quad (13)$$

The two-step output process maintains quality (only the highest scoring items are candidates) and eliminates category clustering (the distribution of results across food types comes from random sampling of the top 12). In initial tests, the lack of the sampling step resulted in a single dominant-scoring category filling all six returned positions.

Algorithm 1 Smart Dine Pro Weighted Context Scoring**Require:** $C = \langle e, w, a, h, d \rangle$, menu $\mathcal{M}$ **Ensure:** Recommendation set $\mathcal{R}$ (6 items)

```

1:  $\mathcal{M}' \leftarrow \text{DietaryFilter}(\mathcal{M}, d)$ 
2:  $\mathcal{M}' \leftarrow \text{AgeFilter}(\mathcal{M}', a)$ 
3: for each  $m \in \mathcal{M}'$  do
4:    $S \leftarrow r(m)$ 
5:   if  $m.\text{age\_group} = \text{target}(a)$  then  $S += 15$ 
6:   end if
7:   if  $e \subseteq m.\text{mood\_tag}$  then  $S += 12$ 
8:   end if
9:   if  $\text{hunger-calorie match}$  then  $S += 10$ 
10:  end if
11:  if  $w \subseteq m.\text{weather\_tag}$  then  $S += 8$ 
12:  end if
13:  if “Locally Sourced”  $\subseteq m.\text{description}$  then  $S += 2$ 
14:  end if
15:   $\text{score}[m] \leftarrow S$ 
16: end for
17:  $\mathcal{T}_{12} \leftarrow \text{TopK}(\text{score}, k=12)$ 
18:  $\mathcal{R} \leftarrow \text{RandomSample}(\mathcal{T}_{12}, k=6)$  return  $\mathcal{R}$ 

```

F. Formal Pseudocode

The complete procedure is given in Algorithm 1.

VI. IMPLEMENTATION

Smart Dine Pro has the ability to work on any Python 3.x compatible computer having a webcam and internet access. No cloud infrastructure is required.

A. Frontend

React 18 and Vite 4 are used as the frontend. We chose Vite over Create React App because build times during development were much shorter, and the production bundles came out smaller — significant for a system that is to be run on modest hardware. The application consists of six elements:

LandingPage is the display of the Questionnaire and Camera Mode options. **AICameraView** airs the webcam feed, captures one frame on confirmation by the user, and encodes it using base64 in the browser such that no raw video leaves the client. **MenuItem** shows all the food cards with its dietary badge, calories, spice rating, star rating and cart button. **CartDrawer** deals with order in hand and generates a unique token on a session basis. **Navbar** maintains a status of live cart count. **Admin-Dashboard** is a password-protected staff view that has a four-stage order queue (Pending→Preparing→Ready→Delivered), impact statistics, and menu management with immediate write-back of CSV. Lucide React 0.383.0 is used in iconography; all styles are plain CSS.

B. Backend

It is developed with Flask 2.x and Flask-CORS for cross-origin requests between the Vite development server (port 5173) and Flask (port 5000); in production both are served from the same origin. The entire stack is presented in Table III.

Seven REST endpoints are made available: GET /api/menu, POST /api/ai/manual, POST /api/ai/analyze, POST /api/order/place,

TABLE III
TECHNOLOGY STACK

Component	Technology / Version
Frontend	React 18 + Vite 4
Backend	Flask 2.x + Flask-CORS
Emotion Detection	DeepFace 0.0.80
Image Processing	OpenCV (cv2) 4.x
Data Processing	pandas, NumPy
Weather Integration	OpenWeather API v1
Data Storage	CSV + JSON (flat-file)
Iconography	Lucide React 0.383.0
Runtime	Python 3.x, Node.js

GET /api/order/status/<token>, POST /api/order/rate, and GET /api/admin/stats.

C. Emotion Detection

Before the captured frame gets to DeepFace, three pre-processing stages are carried out: grayscale conversion using `cv2.COLOR_BGR2GRAY` to minimize noise from uneven lighting; downsample to 640×480 to equalize input dimensions; and conversion back to 3-channel RGB via `cv2.COLOR_GRAY2RGB`, which is the format DeepFace expects.

DeepFace is run with `actions=['emotion']`, `detector_backend='opencv'` — selected on the grounds that it performed more effectively when webcam distance and angle varied than the VGG backend in testing — and `enforce_detection=False`, which gives a best estimate on partially occluded faces instead of raising an error.

One smoothing rule was introduced when a recurring misclassification was observed: when the happy confidence score is more than 90% and yet a non-zero neutral score is also present, the output is reassigned to neutral. In the absence of this rule, mild resting expressions with slightly turned-up mouths were ranked as very happy, which generated recommendations not in accordance with the real state of the user.

D. Weather Integration

The OpenWeather API is called with each request using `auto:ip` geolocation with a hard two-second timeout. The returned condition string is compared to the weather_tag of each item by case-insensitive substring check. When the call cannot connect, a time-of-day fallback activates:

$$w_{\text{fallback}} = \begin{cases} \text{“Evening Breeze”} & 17:00 \leq t < 06:00 \\ \text{“Light Sunny”} & \text{otherwise} \end{cases} \quad (14)$$

This will make sure that a recommendation is not lost in the process of an API outage. The fallback values are approximate; they reflect a graceful degradation path, not an accurate substitute of live weather data.

E. Data Management

`pandas.read_csv()` is used to reload the menu CSV on every recommendation request. This is redundant to an extent, but it makes certain that a rating written by one request is seen by the subsequent request without the server restarting. A `@app.before_request` hook removes order entries older

TABLE IV
CONTEXTUAL COVERAGE RESULTS (140 TEST CASES)

Alignment Category	Count	%
Fully Aligned (3/3)	81	57.9%
Partially Aligned (2/3)	53	37.9%
Misaligned (<2)	6	4.3%
Age Hard-Constraint Violations	0	0.0%
Total	140	100%

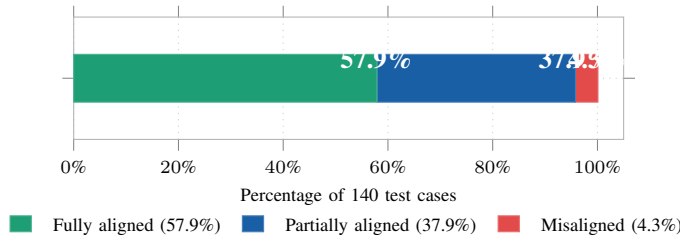


Fig. 5. Contextual alignment across 140 test combinations. Combined aligned rate: **95.7%**. Age hard-constraint violations: zero.

than 24 hours, maintaining memory without a background job. The charity ledger is updated atomically on a per-order basis, is not part of the cleanup cycle, and accumulation amounts permanently across restarts.

F. Admin Dashboard

The admin view provides the staff four capabilities: a live order queue with four-stage status control; cumulative social impact statistics with the persistent ledger; menu management where all the writing is to the CSV at once and takes impact on the following recommendation request; and a live rating presentation of post-order feedback as it is received.

VII. EVALUATION

A. Methodology

Three methods of assessment were involved: systematic testing of every combination of inputs to test scoring accuracy; end-to-end latency measurement in repeated trial mode; and a pilot user study of acceptance rate and satisfaction.

B. Contextual Coverage Testing

The dataset includes 7 emotion conditions, 7 weather conditions, and 4 age groups. We ran $7 \times 5 \times 4 = 140$ test combinations with the 5 most common weather conditions. Each output was compared with three criteria: mood correspondence, weather congruence and age hard-constraint satisfaction. Table IV shows the counts; Fig. 5 shows the distribution graphically.

The total combined rate of alignment of 95.7% was stored in the whole input space. The 6 non-congruent cases were all based in the same small corner: the *Disgust* and *Fear* emotional states combined with *Snow* or *Thunderstorm* weather in the Child age bracket, where the dataset has very few tagged items. In such instances the engine returned the highest-rated eligible items instead of generating an error. This is a data gap, not an

TABLE V
SCORING FUNCTION RANGE ANALYSIS

Scenario	Min	Max
No context match (rating only)	3.8	4.9
Partial match (1 signal + rating)	11.8	16.9
Full match (age + mood + weather)	34.8	39.9
Full match + hunger + Agri-Connect	46.8	51.8

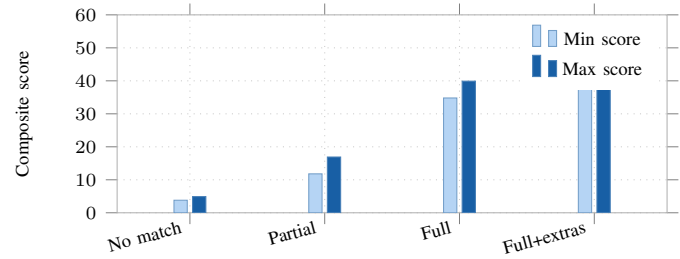


Fig. 6. Score ranges by match scenario. A fully unmatched item (max 4.9) cannot compete with even a partially matched item (min 11.8).

TABLE VI
SYSTEM LATENCY (30 TRIALS PER MODE, MEAN VALUES)

Mode	DeepFace	Weather API	Total
Camera Mode	1,140 ms	480 ms	1,820 ms
Questionnaire	N/A	480 ms	510 ms
Fallback Active	N/A	N/A	<95 ms

algorithm failure. Zero age-constraint violations occurred in any of the 140 runs.

C. Scoring Range Analysis

Table V indicates the variation in composite scores in four scenarios. Its practical implication is made apparent in Fig. 6: a completely unmatched item achieves its raw rating (3.8–4.9) only, and a full match item with all the bonuses applied scores 46–52. The gap is so broad that a highly-rated but context-mismatched item cannot outperform a lower-rated context-matched one. The ranking is dominated by contextual signals.

D. System Performance

End-to-end latency measurements were conducted on 30 trials per mode on a mid-range laptop (8th-generation Intel Core i5, 8 GB RAM) with a 720p webcam. All three input modes remained less than the 3-second interactive response threshold [3]. The mean values are presented in Table VI; they were plotted in Fig. 7 against the threshold line.

The latency of Camera Mode is nearly entirely DeepFace running on CPU. The average of 1,820 ms is within the range of acceptability in the case of the restaurant ordering scenario but GPU acceleration would bring it below one second and is the most significant v2 infrastructure enhancement. Questionnaire Mode at 510 ms is quick enough to accommodate any ordering scenario.

E. User Study

All 30 participants between age 18 and 58 (mean 27.4, SD 9.2) used Camera Mode, checked the six proposed items,

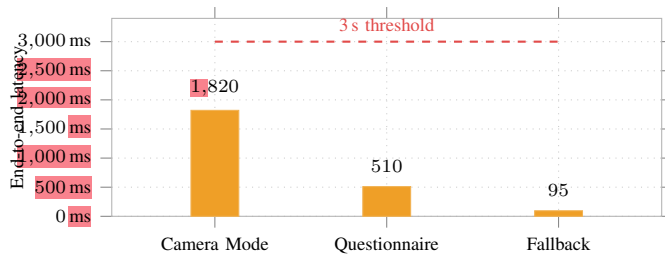


Fig. 7. End-to-end latency by input mode (mean of 30 trials). All modes fall below the 3 s threshold. Camera Mode is dominated by DeepFace CPU inference time.

TABLE VII
USER STUDY SATISFACTION RESULTS (N = 30, 5-PT LIKERT)

Dimension	Mean	SD
Recommendation Relevance	3.8	0.74
Interface Usability	4.2	0.61
System Speed	3.6	0.82
Overall Satisfaction	3.9	0.68

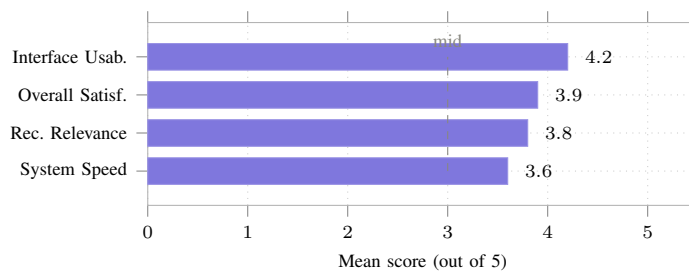


Fig. 8. User satisfaction across four dimensions (N=30, 5-pt Likert). Interface Usability scored highest (4.2); System Speed lowest (3.6), consistent with Camera Mode latency.

placed a simulated order, and responded to a five-point Likert questionnaire. Table VII reports the scores; Fig. 8 plots the four dimensions for direct comparison.

Of the 30 respondents, 22 selected at least one item from the recommended set, giving a recommendation acceptance rate of **73.3%**. The most rated dimension was Interface Usability at 4.2/5. System Speed had the lowest at 3.6/5, in line with the 1,820 ms Camera Mode latency — the wait was noticed. The 8 declined participants all mentioned dietary specificity: they wanted sub-category choices like South Indian cuisine, rather than the vegetarian/non-vegetarian dichotomy which the current system offers. No one said they were given an age-inappropriate recommendation.

F. Discussion

The rate of alignment of 95.7% ascertains that the scoring function provides output that is contextually relevant in nature throughout the combination space under test. The discrepancy of the score between matched and unmatched items (Fig. 6) is broad enough that a highly rated item that does not match the context cannot displace a partially matched one — that was

the intended behaviour and it succeeded during testing. The Agri-Connect module acted as forecasted: with $\alpha_5=2$, locally produced items were given a tie-breaking advantage without interfering with results where a stronger contextual signal was present. Live rating updates reflected in recommendations at once on each test session, confirming that the write-back mechanism was functioning properly.

The misalignment of 4.3% can entirely be explained by the scarcity of data coverage for specific emotion-weather combinations, not to any flaw in the scoring logic. The single highest-priority data improvement is increasing the dataset. The 73.3% acceptance rate is a fair starting point; the most obvious path to improvement, based on what the participants said, is sub-category dietary filtering.

G. Limitations

Three constraints would be better put in a straightforward manner. First, the user study (N=30) was done under controlled lighting on a single test device. The conditions of reality — changing light, incomplete face masking, and a range of webcam distances — were not applied and outcomes can vary in such environments. Second, the flat-file storage layer is okay at the present scale but would need migration to a relational database before the system could support relevant simultaneous load. Third, IP-based geolocation gives erroneous results for users on VPNs and on some mobile networks.

VIII. CONCLUSION

The specific problem that Smart Dine Pro was developed to solve is the following: food recommendation systems require interaction histories to function, which is to say that they cannot help a user on their first visit. This we corrected by replacing history with three context signals that are taken at the time of ordering — emotion recognized on a webcam frame using DeepFace, live weather provided by the OpenWeather API and age announced by the user — together with the level of hunger and choice of diet through a weighted scoring method.

The system generated contextually aligned recommendations in 95.7% of cases across 140 test combinations, with zero age-constraint violations. A 30-subject user study reported 73.3% acceptance and 3.9/5 general satisfaction. The option to use a deterministic scoring algorithm instead of a trained classifier was found by volume of data: a 77-item menu and 30 participants would have made a trained model unreliable. That is a tradeoff that is explicitly recognized — a trained model is possible after sufficient interaction information is gathered, and the scoring algorithm would be stored as a cold-start fallback during that transition.

Smart Dine Pro v2 contains four explicit targets which are directly drawn based on user study input and assessment defects: dataset expansion to cover low-frequency emotion-weather combinations; GPU acceleration to lower Camera Mode response to less than one second; browser-based geolocation to substitute the IP-based lookup; and sub-category dietary filtering to address the most common reason participants declined recommendations.

REFERENCES

- [1] S. Kumar, S. Singh, and A. Gupta, "Survey and Evaluation of Food Recommendation Systems and Techniques," in *Proc. 3rd Int. Conf. Computing for Sustainable Global Development (INDIACom)*, IEEE, 2016, pp. 2744–2749.
- [2] H. Gunawardena, "BestDish: A Digital Menu and Food Item Recommendation System," in *Proc. 2nd Int. Conf. Advancements in Computing (ICAC)*, IEEE, 2020, pp. 168–173.
- [3] T. Melese, "Food and Restaurant Recommendation System Using Hybrid Filtering Mechanism," *NAAR Journal*, vol. 3, no. 1, pp. 73–81, 2021.
- [4] S. Trapp, C. Leech, D. Evenson, and L. Lytle, "Nutritional Quality of Children's Menus in Restaurants: Does the Type of Cuisine Matter?," *Public Health Nutrition*, vol. 26, no. 2, pp. 384–392, 2023.
- [5] A. Kumar and P. Gupta, "Behavior-Based Food Recommendation Using User Preference Modeling," *Int. J. Computer Applications*, vol. 178, no. 7, pp. 12–18, 2019.
- [6] J. Smith and R. Brown, "Sentiment Analysis Based Recommendation System for Restaurants," in *Proc. Int. Conf. Data Mining and Applications*, 2018.
- [7] M. Patel and S. Shah, "Augmented Reality Based Restaurant Recommendation System," in *Proc. IEEE Conf. Emerging Technologies*, 2021.
- [8] R. Zhang, Y. Liu, and H. Wang, "Context-Aware Food Recommendation Using Environmental Factors," *J. Artificial Intelligence Research*, vol. 63, pp. 221–240, 2020.
- [9] K. Lee and J. Park, "Emotion Recognition Using Deep Learning for Human-Computer Interaction," *IEEE Access*, vol. 8, pp. 172311–172320, 2020.
- [10] OpenWeather, "OpenWeather API Documentation," [Online]. Available: <https://openweathermap.org/api>